

Práctico AYED1 - Sala 2 - 14-10

Práctico 5 - Ejercicio 3

Escriba un programa que sume, por un lado, los valores pares y, por otro, los valores múltiplos de 3 de los números entre N y M (inclusivos).

Especificación:

Const N, M : Int; -- estos son los datos de entrada o parámetros del problema
Var sumPar, sumMul3; -- al especificar, sólo definimos las variables que calculan resultados
{ P : $N \leq M$ }

S

{ Q : sumPar = $\langle \sum i : N \leq i \leq M \wedge \text{par}.i : i \rangle \wedge \text{sumMul3} = \langle \sum i : N \leq i \leq M \wedge \text{mul3}.i : i \rangle$ }

donde:

~~par.i = 2 * i~~ NO!!

~~par.i = (2 * i = i)~~ TAMPOCO!!

par.i = (i mod 2 = 0) ESTE SÍ

mul.i = (i mod 3 = 0)

Solución 1:

Const N, M : Int;

Var sumapar, sumamult3, i : Int;

sumapar, sumamult3, i := 0, 0, N;

do i ≤ M →

if ((i mod 2 = 0) ∧ (i mod 3 ≠ 0)) → -- i es par pero no es múltiplo de 3

```

    sumapar:=sumapar+i;
[] ((i mod 3 = 0) ∧ (i mod 2 ≠ 0)) → -- i no es par pero es multiplo de 3
    sumamult3:=sumamult3+i;
[] ((i mod 2 = 0) ∧ (i mod 3 = 0)) → -- i es par y es multiplo de 3
    sumapar:=sumapar+i;
    sumamult3:=sumamult3+i;      -- podría ser asignación multiple
[] ¬((i mod 2 = 0) ∨ (i mod 3 = 0)) → -- i no es par ni multiplo de 3
    skip
fi ;
i:=i+1
od

```

Estamos chequeando dos cosas distintas, 4 combinaciones posibles.

¿ Si tuviera 3 cosas para chequear, cuántas ramas tendría este if ?

Tendría 8, porque son todas las combinaciones posibles de los valores de verdad de 3 cosas.

Mejor es usar if's separados (y secuenciados, uno después del otro) para cada condición.

En seguida lo vemos.

Solución 2:

Const N, M : Int;

Var x, p, i : Int ;

x := N ; p:= 0 ; i:=0 ; -- usar ";" al secuenciar sentencias!!!
 -- o usar asignación múltiple

do x <= M →

 -- (las guardas no son disjuntas pero eso no es un requisito para el if)

 if mod x 2 = 0 → --- x es par (no sé si es multiplo de 3 o no)

 p := p + x --- **si yo entro acá, NO ENTRO EN LAS OTRAS RAMAS.**

- si x es 6, puedo entrar acá y no voy a entrar a la otra rama. MAL.
- lo que necesito es tener dos if distintos, uno después del otro.

```

□ mod x 3 = 0 →
    i := i + x

```

-- error seguro en este if: no contempla el caso en que x no es par ni múltiplo de 3.

```

□ mod x 2 ≠ 0 ∧ mod x 3 ≠ 0 →
    skip

```

```

fi ;

```

```

x = x + 1      --- acá no va ";"

```

```

do

```

Hacemos un paréntesis para hablar del ";": El ";" me arma una nueva sentencia "S;T" que resulta de secuenciar dos sentencias S y T.

¿Cómo es el do? Es "do B → U od" a donde U es una sentencia. Si U es una secuenciación "S;T" me queda: "do B → S ; T od" (o sea, ver que no hay ";" al final).

Solución 3:

```

Const N, M: Int;

```

```

Var i, p, t: int;

```

```

p, t, i := 0, 0, N ;      --- faltaba inicializar i

```

```

do N ≤ i ∧ i ≤ M →

```

```

    if (i mod 2 = 0 ∧ i mod 3 = 0) ->

```

```

        p:=p+i ; t:=t+i

```

```

    □ (i mod 2 = 0) ->  -- problema acá, podría ser que i sea múltiplo de 3. falta "∧ i mod 3 ≠ 0".

```

```

        p:=p+i

```

```

    if (i mod 3 = 0) -> -- problema acá, podría ser que i sea múltiplo de 3. falta "∧ i mod 2 ≠ 0".
        t:=t+i
    if (i mod 2 ≠ 0 ∧ i mod 3 ≠ 0) ->
        skip
    fi ;
    i := i + 1
od

```

Solución 4:

```

Const N, M : Int;
Var x, y, i : Int; -- en x sumaremos los pares, en y los multiplos de 3
x, y := 0, 0 ; i := N ;
do (N <= i <= M) → -- acá es "N ≤ i ∧ i ≤ M".
    -- supongamos que i = 6. en este caso, es par y multiplo de 3,
    -- entonces quiero sumarlo tanto a x como a y.
    -- quiero que entre al mismo tiempo a dos ramas distintas de un solo if que hice.
    -- IMPOSIBLE ya que siempre en el if se entra sólo a una de las ramas.
    if mod i 2 = 0 →
        x := x + i ;
        i := i+1
    ; y:=y -- esto no hace nada, no hace falta
    if mod i 3 = 0 →
        y := y + i ;
        i := i+1
    ; x:=x -- esto no hace nada, no hace falta
    if mod i 2 ≠ 0 ∧ mod i 3 ≠ 0 →
        i := i+1
    fi
do

```

Solución 5:

Cons N,M: Int;

Var sumpares, summultiplos : Int;

sumpares := 0; summultiplos:=0 ; k:=N ;

do (k ≤ M) →

do (par.k) →

-- acá si vale par.k me quedo para siempre en este ciclo.

sumpares := sumpares + k

od ;

do (multiplo3.k) →

-- lo mismo, puedo quedarme infinitamente acá

summultiplos := summultiplos + k

od ;

k := k+1

od

“;” funciona como un operador de sentencias, como “+” es un operador de números, o “++” es un operador de listas:

“4 + 5 + ”

“S ; T ;”

“xs ++ ys ++”

Solución auspiciada por la cátedra:

Const N, M : Int;

Var i, sumPar, sumMul3;

{ P : N ≤ M }

i, sumPar, sumMul3 := N, 0, 0 ;

do i ≤ M →

if par.i →

sumPar := sumPar + i

□ ¬ par.i →

skip

```

fi ;
if mul3.i  $\rightarrow$ 
    sumMul3 := sumMul3 + i
 $\square \neg$  mul3.i  $\rightarrow$ 
    skip
fi ;
i := i + 1
od
{ Q : sumPar =  $\langle \sum i : N \leq i \leq M \wedge \text{par}.i : i \rangle \wedge$  sumMul3 =  $\langle \sum i : N \leq i \leq M \wedge \text{mul3}.i : i \rangle$  }

```

Práctico 5 - Ejercicio 2: Factorial

Solución posible:

```

Cons N: Int;
Var i, n : Int;    --- en i se calcula el resultado
i := 1 ; n := 0 ;
do n  $\leq$  N  $\rightarrow$   -- acá si ponemos n < N el programa es correcto (pero molesto).
    n := n + 1 ; -- bien al principio, cuando n = 0, pero mal al final porque si n = N, le suma 1 y termina multiplicando N+1.
    i := i * n
od

```

Solución auspiciada por la cátedra:

```

Const N : Int;
Var fac, i : Int;
fac, i := 1, 0;
do i < N →
    fac, i := fac * (i + 1), i + 1
od

```

Soluciones que calculan en otro orden:

```

Const N : Int;
Var fac, i : Int;
fac := 1; i := N;
do 1 ≤ i ->
    fac := fac * i;
    i := i - 1      -- acá podría usar asignación múltiple
od

```

Asignación múltiple: ¿Cuáles de las siguientes son equivalentes?

A: fac := fac * i;
i := i - 1

B: i := i - 1

fac := fac * i; -- acá es usa para i el nuevo valor

C: fac, i := fac * i, i - 1

D: i, fac := i - 1, fac * i

Lo más obvio: A != B.

Un poco menos obvio: C = D.

Ahora, ¿ $A = C = D$ o $B = C = D$?

Repaso de la asignación múltiple:

Supongamos que tenemos la asignación:

$x_1, x_2, \dots, x_n := E_1, E_2, \dots, E_n$

Esta asignación quiere decir que:

1. Primero, se calculan los valores de todas las expresiones $E_1, E_2, \dots, E_n$, usando el estado anterior.
2. Luego, se modifica el estado asignando a cada variable $x_1, x_2, \dots, x_n$ el valor nuevo que le corresponde.

Viendo esto podemos concluir que $A = C = D \neq B$.

Tarea para la casa: ¿cuáles de las siguientes son equivalentes?

A: $fac := fac * i;$
 $i := fac - 1$

B: $i := fac - 1$
 $fac := fac * i$

C: $fac, i := fac * i, fac - 1$

D: $i, fac := fac - 1, fac * i$

Podría suceder que ni A ni B son equivalentes a C y D.

Otra: Comparar las siguientes:

$x, y := y, x$

$x := y ; y := x$

$y := x ; x := y$